

THIẾT KẾ KỸ THUẬT

1. Kiến trúc tổng thể

Hệ thống UniHub được thiết kế theo kiến trúc Client-Server kết hợp Background Processing.

- Lý do lựa chọn: Do giới hạn thời gian (2 tuần), kiến trúc Microservices là quá phức tạp và tốn thời gian thiết lập hạ tầng. Thay vào đó, một Backend API tập trung (Node.js) xử lý toàn bộ request realtime kết hợp với một Worker chạy ngầm sẽ đảm bảo API không bị nghẽn khi phải xử lý các tác vụ nặng (đọc CSV, gọi AI, gửi email).
- Xử lý sự cố: Nếu Worker (xử lý nền) gặp sự cố (ví dụ crash khi đọc file CSV lỗi), hệ thống API chính vẫn sống và sinh viên vẫn đăng ký workshop bình thường. Nếu Cổng thanh toán (hệ thống ngoài) chết, hệ thống áp dụng Graceful Degradation (Circuit Breaker) để không làm treo giao diện.

2. Sơ đồ C4 (C4 Diagram)

a) Cấp 1 - Ngữ cảnh hệ thống (Level 1 - System Context)

(UniHub nằm ở đâu giữa bức tranh tổng thể, ai dùng nó và nó gọi đi đâu)

[Mô tả để vẽ sơ đồ]

- Hệ thống trung tâm: [System] UniHub Workshop
- Người dùng (Actors) trở mũi tên vào UniHub:
 - + [Person] Sinh viên: Xem lịch, đăng ký, nhận vé QR.
 - + [Person] Ban tổ chức: Quản lý sự kiện, xem thống kê.
 - + [Person] Nhân sự Check-in: Quét mã QR điểm danh.
- Các hệ thống bên ngoài (External Systems) mà UniHub trở mũi tên tới:
 - + [System] Legacy Student System: Cung cấp file CSV dữ liệu sinh viên.
 - + [System] Mock Payment Gateway: Xử lý giao dịch ảo và trả về kết quả.
 - + [System] AI Provider (Gemini/OpenAI): Nhận text PDF & trả tóm tắt.
 - + [System] Mock Notification (Mailtrap): Nhận yêu cầu gửi email xác nhận.

graph TD

%% Định nghĩa các Style

classDef core fill:#1168bd,stroke:#0b4884,color:#fffff,rx:5px,ry:5px;

classDef external fill:#999999,stroke:#666666,color:#fffff,rx:5px,ry:5px;

classDef actor fill:#08427b,stroke:#052e56,color:#fffff;

%% Khai báo Actors

Student(("Sinh viên")):::actor

Organizer(("Ban tổ chức")):::actor

Staff(("Nhân sự Check-in")):::actor

%% Khai báo Hệ thống

UniHub["[System]\nUniHub Workshop"]:::core

%% Khai báo Các hệ thống bên ngoài (Đã thêm tiếng Việt)

LegacySys["[External]\nLegacy Student System\n(Hệ thống quản lý SV cũ)"]:::external

Payment["[External]\nMock Payment Gateway\n(Cổng thanh toán giả lập)"]:::external

AI["[External]\nAI Provider (Gemini)\n(Dịch vụ cung cấp AI)"]:::external

Notify["[External]\nMock Notification\n(Hệ thống thông báo giả lập)"]:::external

%% Kết nối

Student -->|Xem lịch, đăng ký, vé QR| UniHub

Organizer -->|Quản lý, thống kê| UniHub

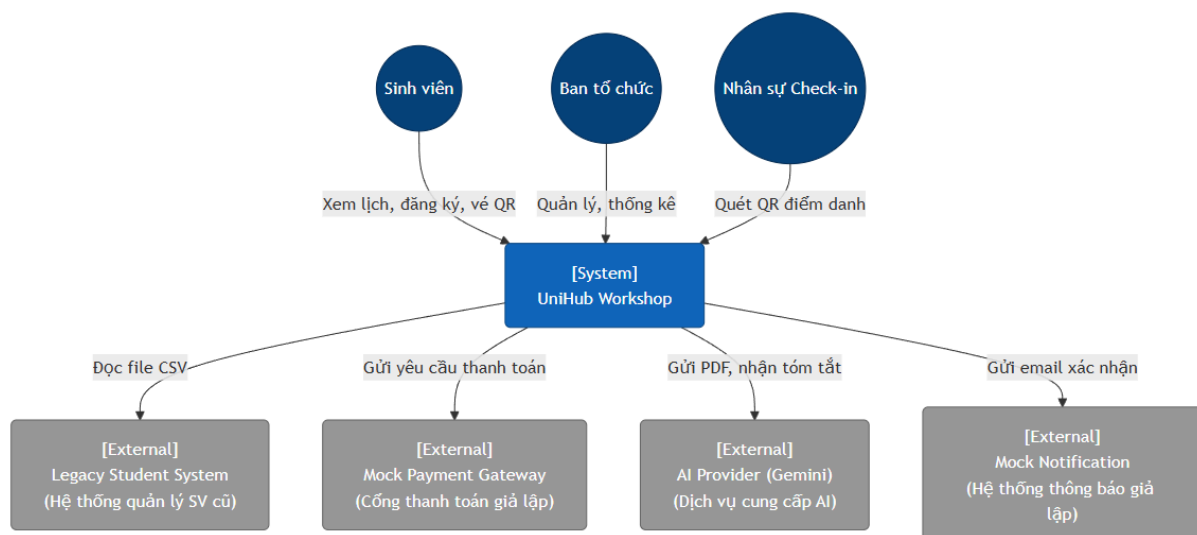
Staff -->|Quét QR điểm danh| UniHub

UniHub -->|Đọc file CSV| LegacySys

UniHub -->|Gửi yêu cầu thanh toán| Payment

UniHub -->|Gửi PDF, nhận tóm tắt| AI

UniHub -->|Gửi email xác nhận| Notify



b) Cấp 2 - Các thành phần triển khai (Level 2 - Container)

(Phân rã hệ thống UniHub thành các khối công nghệ cụ thể)

[Mô tả để vẽ sơ đồ] Khối hệ thống UniHub Workshop ở Level 1 được chia ra thành 6 Container giao tiếp với nhau:

- [Container] Web Application (ReactJS): Single Page Application phục vụ Sinh viên và Ban tổ chức. Giao tiếp với Backend qua REST API.
- [Container] Mobile Application (Native Android): App dành riêng cho nhân sự. Việc triển khai ứng dụng Android native kết hợp với Room Database sẽ giúp thiết lập kiến trúc Offline-first một cách vững chắc, thao tác truy xuất dữ liệu cục bộ và đồng bộ hai chiều với server sẽ diễn ra trơn tru.
- [Container] Core Backend API (Node.js/Express): Nhận HTTP request, xử lý phân quyền, thực hiện Rate Limiting và kiểm soát Concurrency Lock.
- [Container] Background Worker (Node.js): Process chạy độc lập. Lắng nghe Message Queue để gọi AI, xử lý thông báo và chạy Cronjob đọc file CSV mỗi đêm.
- [Container] Primary Database (PostgreSQL): RDBMS lưu trữ dữ liệu lõi (Users, Workshops, Registrations) với tính toàn vẹn ACID cao nhất.
- [Container] Cache & Broker (Redis): In-memory database làm 3 nhiệm vụ chính: Đếm request (Rate limit), Giữ khóa phân tán (Distributed Lock chống tranh chấp), và Message Queue để chuyển việc từ API sang Worker.

graph TD

%% Định nghĩa các Style theo chuẩn C4 (Tone xanh dương)

classDef container fill:#438dd5,stroke:#3c7fc0,color:#ffffff,rx:5px,ry:5px;

classDef database fill:#2b7fb2,stroke:#2672a0,color:#ffffff;

classDef actor fill:#08427b,stroke:#052e56,color:#ffffff;

%% Khai báo Actors

Student(("Sinh viên")):::actor

Organizer(("Ban tổ chức")):::actor

Staff(("Nhân sự Check-in")):::actor

%% Khai báo Ranh giới hệ thống (System Boundary)

subgraph "UniHub Workshop System"

%% Khai báo các Containers (Kèm tiếng Việt)

WebApp["[Container: ReactJS]\nWeb Application\n(Giao diện SPA cho SV & Ban tổ chức)"]:::container

MobileApp["[Container: Android + Room]\nMobile Application\n(App quét QR điểm danh Offline-first)"]:::container

API["[Container: Node.js/Express]\nCore Backend API\n(Xử lý Logic lõi, Phân quyền, Lấy Lock)"]:::container

Worker["[Container: Node.js]\nBackground Worker\n(Chạy ngầm: Xử lý CSV, AI, Email)"]:::container

DB["[Container: PostgreSQL]\nPrimary Database\n(Lưu trữ ACID: User, Workshop, Vé)"]:::database

Redis["[Container: Redis]\nCache & Broker\n(Lưu Hàng đợi, Rate Limit, Khóa chặn)"]:::database

end

%% Tương tác từ User đến Hệ thống

Student -->|HTTPS: Đăng ký, tải QR| WebApp

Organizer -->|HTTPS: Quản lý, thống kê| WebApp

Staff -->|Quét QR lưu offline và Sync online| MobileApp

%% Tương tác giữa các Container bên trong

WebApp -->|REST API JSON/HTTPS| API

MobileApp -->|Đồng bộ dữ liệu 2 chiều| API

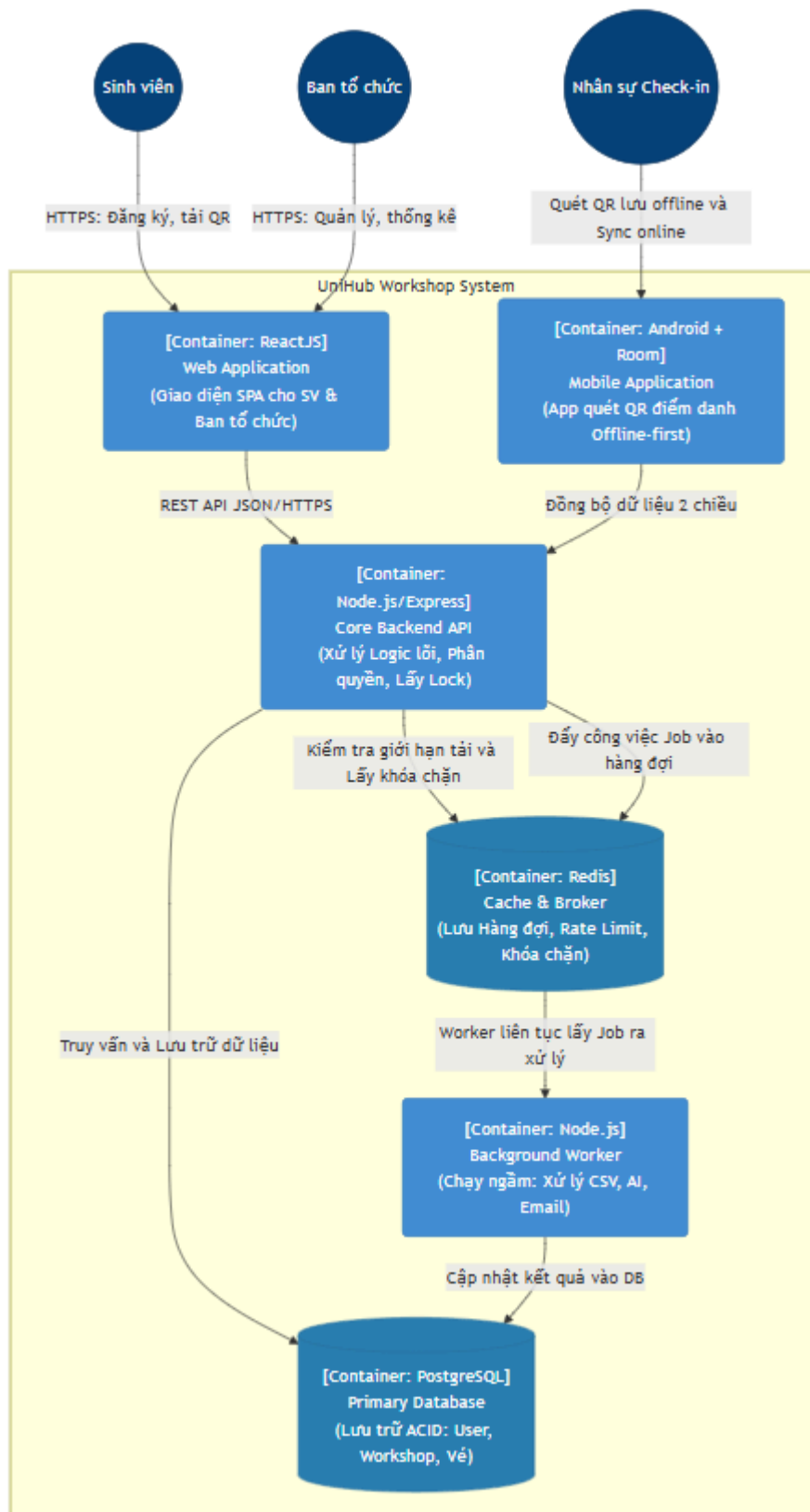
API -->|Kiểm tra giới hạn tài và Lấy khóa chặn| Redis

API -->|Truy vấn và Lưu trữ dữ liệu| DB

API -->|Đẩy công việc Job vào hàng đợi| Redis

Redis -->|Worker liên tục lấy Job ra xử lý| Worker

Worker -->|Cập nhật kết quả vào DB| DB



3. High-Level Architecture Diagram

(Sơ đồ tập trung vào luồng dữ liệu - Data Flow)

[Mô tả để vẽ sơ đồ]

- **Cụm Client:**
 - + React Web gửi request qua HTTPS tới Node.js API.
 - + Android App gửi/nhận dữ liệu JSON đồng bộ tới Node.js API. Khi rớt mạng, app đọc/ghi trực tiếp xuống SQLite Local DB. Khi có mạng lại, dữ liệu sẽ được đồng bộ từ SQLite tới Node.js.
- **Cụm Server:**
 - + Mọi request từ Client đều chạm vào Node.js API.
 - + Ngay tại cửa ngõ API, request bị kiểm tra bởi Redis (Rate Limiter). Nếu lớn hơn ngưỡng => Trả về lỗi 429 Too Many Requests.
 - + Khi có request đăng ký, Node.js API gọi Redis để lấy Lock, sau đó mới ghi xuống PostgreSQL.
 - + Khi đăng ký thành công, Node.js API ném một “Task” (Job) vào Redis Queue.
- **Cụm Background & Tích hợp:**
 - + Node.js Worker liên tục bốc “Task” từ Redis Queue ra.
 - + Worker gọi API của Mock Payment Gateway (nếu là vé có phí).
 - + Worker gọi hệ thống Notification để nhả email.
 - + Đến đêm, Worker tự kích hoạt chạy script đọc file CSV từ Local Disk (mô phỏng Legacy System) rồi cập nhật vào PostgreSQL.

graph LR

%% Định nghĩa các Style

classDef db fill:#f39c12,stroke:#e67e22,color:#ffffff,rx:5px,ry:5px;

classDef api fill:#2ecc71,stroke:#27ae60,color:#ffffff,rx:5px,ry:5px;

classDef worker fill:#9b59b6,stroke:#8e44ad,color:#ffffff,rx:5px,ry:5px;

classDef client fill:#3498db,stroke:#2980b9,color:#ffffff,rx:5px,ry:5px;

%% 1. Cụm Client

subgraph Client ["Cụm Client"]

React["Web App ReactJS"]:::client

Android["Mobile App Android"]:::client

end

%% 2. Cụm Server

subgraph Server ["Cụm Server (Realtime)"]

Gateway["Node.js Core API"]:::api

RedisRateLimit["Redis\nRate Limit và Lock"]:::db

PostgresDB["PostgreSQL\nPrimary DB"]:::db

end

%% 3. Cụm Background

subgraph Background ["Cụm Background (Chạy ngầm)"]

RedisQueue["Redis\nMessage Queue"]:::db

WorkerNode["Node.js Worker"]:::worker

External["Các hệ thống ngoài\nPayment AI Email"]:::client

CSV["File CSV sinh viên\nLegacy System"]:::db

end

%% Luồng 1: Tương tác Client - Server

React -->|1 HTTPS Request| Gateway

Android <-->|1 Sync Offline Data| Gateway

%% Luồng 2: Xử lý tại Server

Gateway -->|2 Check Limit và Lấy Lock| RedisRateLimit

Gateway -->|3 Read và Write dữ liệu| PostgresDB

Gateway -->|4 Push Task vào hàng đợi| RedisQueue

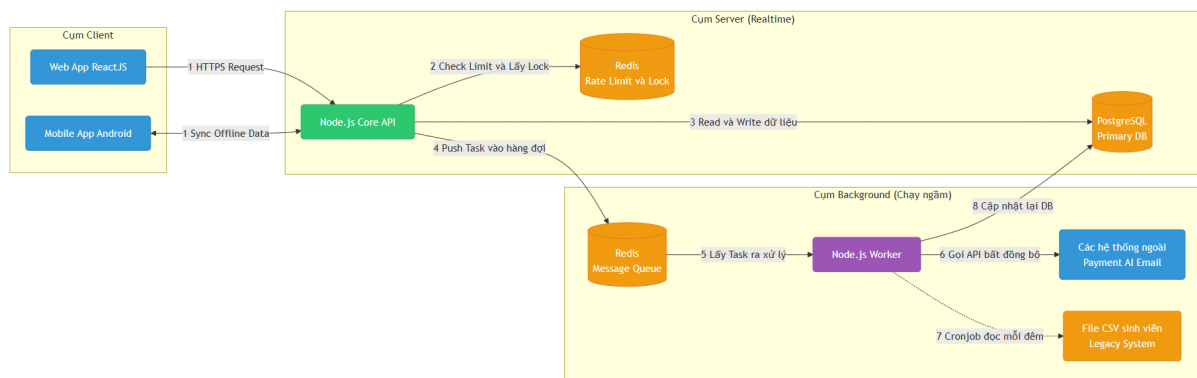
%% Luồng 3: Xử lý ngầm

RedisQueue -->|5 Lấy Task ra xử lý| WorkerNode

WorkerNode -->|6 Gọi API bất đồng bộ| External

WorkerNode -->|7 Cronjob đọc mỗi đêm| CSV

WorkerNode -->|8 Cập nhật lại DB| PostgresDB



4. Thiết kế cơ sở dữ liệu

a) Hệ thống sử dụng mô hình đa cơ sở dữ liệu (Polyglot Persistence) để tối ưu cho từng bài toán cụ thể:

- PostgreSQL (Database chính): Chọn cơ sở dữ liệu quan hệ (RDBMS) vì nghiệp vụ đăng ký workshop đòi hỏi tính toàn vẹn dữ liệu (ACID) cực kỳ cao. Khi xử lý trừ số lượng available_slots, các transaction của SQL kết hợp với cơ chế khóa (Lock) sẽ ngăn chặn tuyệt đối tình trạng overbooking.
- Redis (In-memory Database): Sử dụng làm bộ nhớ đệm (Cache) để giảm tải cho DB chính khi hàng ngàn sinh viên cùng load danh sách workshop. Đồng thời dùng để lưu trữ trạng thái Rate Limiting, Idempotency Key và làm Message Broker (hàng đợi xử lý bất đồng bộ).
- SQLite / Room (Local DB): Lưu trữ dữ liệu danh sách mã QR trên thiết bị Android của nhân sự để phục vụ luồng check-in offline.

b) Thiết kế Schema cho các Entity quan trọng nhất (PostgreSQL):

- Users: id (PK), email, full_name, role (STUDENT, ORGANIZER, CHECKIN_STAFF), created_at.
- Workshops: id (PK), title, description, room_name, total_slots, available_slots, is_paid, price, start_time, end_time. (Cột available_slots sẽ được đánh Index để truy vấn nhanh).
- Registrations: id (PK), user_id (FK), workshop_id (FK), qr_code_hash, payment_status (PENDING, PAID, FAILED), checkin_status (BOOLEAN), checkin_time.
- Payments: id (PK), registration_id (FK), idempotency_key (UNIQUE), amount, status, created_at.

5. Thiết kế kiểm soát truy cập

Hệ thống sử dụng mô hình RBAC (Role-Based Access Control) kết hợp với JWT (JSON Web Token) để phân quyền và xác thực người dùng.

- Xác thực (Authentication): Sau khi đăng nhập, hệ thống cấp 1 Access Token (thời hạn ngắn, 15 phút) lưu ở bộ nhớ tạm của Frontend và 1 Refresh Token (thời hạn dài, 7 ngày) lưu trong HttpOnly Cookie để chống tấn công XSS.
- Phân quyền (Authorization): Định nghĩa 3 Role với quyền hạn thiết lập chặt chẽ tại Middleware của Backend API:
 - + STUDENT: Chỉ có quyền GET /workshops, POST /registrations (áp dụng cho chính user_id đó) và GET /registrations/my-tickets.

- + ORGANIZER: Có quyền truy cập các endpoint nội bộ POST/PUT/DELETE /workshops, xem thống kê GET /stats, và trigger API AI Summary.
- + CHECKIN_STAFF: Chỉ có quyền gọi API đồng bộ danh sách QR GET /sync-data và cập nhật trạng thái điểm danh PUT /registrations/checkin.

6. Thiết kế các cơ chế bảo vệ hệ thống

a) Kiểm soát tải đột biến (Spike Load Handling)

- Giải pháp: Sử dụng thuật toán Token Bucket thực thi trên Redis.
- Cách hoạt động: Giới hạn mỗi User/IP chỉ được phép gửi tối đa 30 requests / phút. Nếu vượt quá (ví dụ bot spam hoặc user bấm F5 liên tục), API Gateway chặn ngay lập tức và trả về HTTP Status 429 Too Many Requests.
- Bỏ trợ: Với 12.000 sinh viên truy cập đồng thời, giao diện “Xem lịch workshop” sẽ được Cache thẳng trên Redis. API sẽ trả về dữ liệu từ RAM thay vì xuống PostgreSQL, giúp hệ thống không bị crash ở những phút đầu tiên.

b) Xử lý công thanh toán không ổn định

- Giải pháp: Triển khai Design Pattern Circuit Breaker (Cầu dao tự động) kết hợp Graceful Degradation (Suy giảm duyên dáng).
- Cách hoạt động: Khi API gọi sang Cổng thanh toán (Mock), nếu tỷ lệ lỗi (timeout/500) vượt ngưỡng 50% trong 10 giây, Circuit Breaker sẽ chuyển sang trạng thái OPEN.
- Hành vi khi lỗi: Lúc này, mọi request thanh toán mới sẽ bị chặn ngay ở backend (Fail-fast) trả về lỗi mượt mà cho client (“Cổng thanh toán đang bảo trì, vui lòng thử lại sau 5 phút”) thay vì treo hệ thống. Đặc biệt (Graceful Degradation), database và các API khác vẫn hoạt động bình thường, sinh viên vẫn có thể lướt xem workshop hoặc đăng ký các workshop *Miễn phí*.

c) Chống trùng tiền hai lần (Idempotency)

- Giải pháp: Sử dụng cơ chế Idempotency Key.
- Cách hoạt động:
 - + Khi sinh viên bấm “Thanh toán”, Frontend sinh ra một UUID ngẫu nhiên (Vd: key_123) và gắn vào Header X-Idempotency-Key của request.
 - + Backend nhận được request, sẽ kiểm tra key này trong Redis.
 - + Nếu key CHƯA tồn tại: Thực hiện trừ tiền, lưu trạng thái thành công và gắn key_123 vào Redis (với TTL hết hạn sau 24h).

- + Nếu cổng thanh toán chập chờn, mạng lag, sinh viên bấm nút thanh toán thêm 3 lần nữa (cùng gửi lên key_123). Backend thấy key này ĐÃ có trong Redis, sẽ lập tức trả về kết quả “Thanh toán thành công” của lần xử lý trước đó mà không thực hiện gọi cổng thanh toán lần 2. Đảm bảo an toàn tuyệt đối tài chính.

7. Thiết kế các cơ chế bảo vệ hệ thống (ADR - Architecture Decition Records)

- ADR 1: Chọn PostgreSQL thay vì MongoDB làm Database chính
 - + *Đánh đổi:* MongoDB linh hoạt hơn về cấu trúc, nhưng hệ thống UniHub có yêu cầu rất khắt khe về việc trừ số lượng chỗ trống (available_slots). PostgreSQL hỗ trợ cơ chế Locking mạnh mẽ (Pessimistic/Optimistic Lock) và Transaction ACID, giúp giải quyết triệt để bài toán tranh chấp chỗ ngồi, điều mà NoSQL xử lý khó khăn hơn.
- ADR 2: Sử dụng Redis làm Message Broker (với BullMQ) thay vì Kafka/RabbitMQ
 - + *Đánh đổi:* Kafka/RabbitMQ là những hệ thống Message Queue chuyên dụng và mạnh mẽ cho production. Tuy nhiên, với nhân lực 2 người và thời gian 2 tuần, việc setup hạ tầng Kafka là lãng phí. Redis có sẵn tốc độ cực cao, lại đang được dùng cho Cache/Rate Limit, nên tích hợp thêm thư viện BullMQ trên Node.js để chạy Background Worker (gửi email, gọi AI, đọc CSV) là quyết định tối ưu nguồn lực nhất.
- ADR 3: Thiết kế App Native Android cho luồng Check-in thay vì Web App
 - + *Đánh đổi:* Cần code thêm một nền tảng mới (Android/Java) thay vì dùng chung ReactJS. Tuy nhiên, trình duyệt Web (kể cả PWA) đôi khi không ổn định trong việc lưu trữ offline khi bộ nhớ điện thoại đầy (bị hệ điều hành tự động dọn Cache). Ứng dụng Android kết hợp cơ sở dữ liệu cục bộ (Room SQLite) đảm bảo dữ liệu điểm danh offline được lưu trữ bền vững, an toàn và đồng bộ hóa (sync) chính xác 100% khi có mạng lại.